

State Design Pattern

Intent

Let an object **change its behavior when its internal state changes**, so it appears to change its class. Instead of one object with a big conditional (if `state == RED` ... else if `state == GREEN` ...), you give each state its **own class**, and the object *delegates* its behavior to whichever state object it currently holds. Each state also knows **which state comes next**, so the transitions live in the states themselves.

Here the object is a **traffic light**. It behaves differently depending on whether it's red, green, or yellow — and each colour knows what colour follows it: red → green → yellow → red.

UML class diagram

```
<<interface>> ITrafficLightState | +handleRequest(TrafficLightContext) | ^ ^ ^ | | |
RedLightState GreenLightState YellowLightState "stop" "go" "slow down" | | | +--- each
sets context.setState(NEXT) ---+ RED -> GREEN -> YELLOW -> RED +-----+
TrafficLightContext | | -state | | +setState(state) | | +request() -----+-->
state.handleRequest(this) +-----+
```

The players

```
states/ITrafficLightState the state contract: handleRequest(context)
states/concrets/RedLightState "stop" → then becomes Green /GreenLightState "go" → then
becomes Yellow /YellowLightState "slow down" → then becomes Red
context/TrafficLightContext the context: holds the current state, delegates to it
StateDesignPattern the demo – starts Red, fires three requests
```

The code, line by line

ITrafficLightState — the state contract

```
public interface ITrafficLightState { void handleRequest(TrafficLightContext
trafficLightContext); }
```

- One method: `handleRequest(context)` — "act according to *this* state, and then decide what state the context should move to."

- It receives the **context** as a parameter. That's essential: it's how a state is able to push the context into its *next* state (see *Why the context is passed in* below).

The concrete states

```
public class RedLightState implements ITrafficLightState { @Override public void
handleRequest(TrafficLightContext c) { System.out.println("Red Light: Cars must stop.");
c.setState(new GreenLightState()); // transition: Red → Green } } public class
GreenLightState implements ITrafficLightState { @Override public void
handleRequest(TrafficLightContext c) { System.out.println("Green Light: Cars can go.");
c.setState(new YellowLightState()); // transition: Green → Yellow } } public class
YellowLightState implements ITrafficLightState { @Override public void
handleRequest(TrafficLightContext c) { System.out.println("Yellow Light: Cars must slow
down to stop."); c.setState(new RedLightState()); // transition: Yellow → Red } }
```

Each state class does exactly **two things** inside `handleRequest`:

1. **The behavior for that state** — the `System.out.println(...)` line is what the traffic light *does* while it's this colour.
2. **The transition** — `c.setState(new ...())` tells the context which state to become next. This is the key idea: **each state owns its own outgoing transition**. Red knows the next state is Green; Green knows it's Yellow; Yellow knows it's Red. The "what comes next" logic is distributed across the states, not centralized in the context.

Because each state points to the next, the three classes form a **cycle**: Red → Green → Yellow → Red → ...

TrafficLightContext — the context

```
public class TrafficLightContext { private ITrafficLightState nextITrafficLightState;
public void setState(ITrafficLightState nextITrafficLightState) {
this.nextITrafficLightState = nextITrafficLightState; } public void request() {
nextITrafficLightState.handleRequest(this); } }
```

- `private ITrafficLightState nextITrafficLightState;` — the context holds its **current state** by the *interface* type. It never mentions `RedLightState` etc.; it only knows "I have some state." This is what lets the behavior change without the context changing.
- `setState(...)` — replaces the current state. It's called both from the outside (to set the *initial* state) and from *inside* the states themselves (to perform transitions).
- `request()` — the context's single public action. It **delegates** straight to the current state's `handleRequest(this)`, passing itself along so the state can act *and* transition. The context contributes no behavior of its own — it just forwards to whatever state it's in.

StateDesignPattern — the demo

```
TrafficLightContext context = new TrafficLightContext(); context.setState(new RedLightState()); // start in Red
context.request(); // Red → prints "stop", transitions to Green
context.request(); // Green → prints "go", transitions to Yellow
context.request(); // Yellow → prints "slow down", transitions to Red
```

- The context is seeded with the initial `RedLightState`.
- Each `context.request()` call runs the **current** state's behavior, then the state flips the context to the next colour. So the *same* call, `context.request()`, produces **different output each time** — because the object behind the wheel keeps changing.

Why the design decisions

Why State instead of a big `if/switch` on a status field?

The procedural version looks like:

```
void request() { if (state == RED) { print("stop"); state = GREEN; } else if (state == GREEN) { print("go"); state = YELLOW; } else if (state == YELLOW) { print("slow down"); state = RED; } }
```

Problems: **every** behavior and **every** transition is crammed into one growing method; adding a state means editing that method (and probably several others like it); and the transition rules are tangled together where they're easy to get wrong. State replaces the conditional with **polymorphism** — each `case` becomes a class, and "which branch runs" becomes "which state object is currently held." Adding a state is adding a class, not editing a conditional (Open/Closed Principle).

Why does each state hold the transition to the *next* state?

So the state machine's wiring lives with the states that own it. `RedLightState` is the single source of truth for "what follows red." This keeps each transition **local and cohesive**: to understand or change the red→green rule, you look in exactly one place. The context stays dumb — it doesn't need to know the transition table at all.

(Trade-off: because transitions are distributed, no single file shows the *whole* machine at a glance. When the graph is complex, some implementations instead centralize transitions in the context or a table. This module uses the distributed style, which is the classic GoF form.)

Why is the context's state field typed as the interface?

Same principle as Strategy: **program to an interface, not an implementation**. Holding an `ITrafficLightState` means the context works with any state, present or future, and its behavior changes purely by swapping the referenced object — the context's own code never changes.

Why is the context passed into `handleRequest(this)`?

Because the state needs a way to **change the context's state** — that's the transition. By handing the state a reference to the context, the state can call `context.setState(nextState)` to move the machine forward. Without that back-reference, states couldn't drive transitions and the context would have to contain the transition logic (back to the big conditional).

State vs. Strategy (they look almost identical — here's the difference)

Structurally State and Strategy are twins: a context delegating to an interface-typed object. The difference is **intent and who changes the object**:

- **Strategy** — the *client* picks one algorithm and it usually stays put; strategies don't know about each other. It's about *how* to do something.
- **State (this module)** — the object **transitions between states on its own**, and states *do* know about each other (each sets the next). It's about *what the object is* right now, and it models a state machine that evolves over time.

The tell is the self-transition: `c.setState(new GreenLightState())` inside a state is pure State pattern — a Strategy would never reassign the context's strategy from inside itself.

Execution flow (the demo — three requests)

```
main ■ context.setState(RedLightState) initial state = Red ■ ■■■ context.request() ■  
■■■ RedLightState.handleRequest(context) ■ ■■■ prints "Red Light: Cars must stop." ■  
■■■ context.setState(GreenLightState) ← now Green ■ ■■■ context.request() ■ ■■■  
GreenLightState.handleRequest(context) ■ ■■■ prints "Green Light: Cars can go." ■ ■■■  
context.setState(YellowLightState) ← now Yellow ■ ■■■ context.request() ■ ■■■  
YellowLightState.handleRequest(context) ■ ■■■ prints "Yellow Light: Cars must slow down to  
stop." ■ ■■■ context.setState(RedLightState) ← back to Red
```

Console output:

```
State Design Pattern Red Light: Cars must stop. Green Light: Cars can go. Yellow Light:  
Cars must slow down to stop.
```

Same method called three times, three different outcomes — because the context's internal state advances Red → Green → Yellow on each call. A fourth `request()` would print the Red message

again and the cycle repeats.

Notes / possible extensions (not changed in the code)

- **States are created fresh on each transition** (`new GreenLightState()` etc.). Since these states hold no data, they could be **singletons/shared instances** to avoid allocations — a common optimization when states are stateless.
- **The field name is `nextITrafficLightState`**. It actually holds the *current* state (the one that will run on the next `request()`), so a name like `currentState` would read more naturally — a naming nuance, not a behavioral one (code left as-is).
- **Plain `main`, no Spring** — the pattern is pure OO structure and needs no container.

Relationship to the other Behavioral patterns

- **State (this module)** — an object's behavior changes as it **transitions between states**; states know their successors and drive the transitions.
- **Strategy** — swap **one** algorithm chosen by the client; no self-transitions.
- **Chain of Responsibility** — pass a request along handlers until one handles it.
- **Template Method** — fix an algorithm skeleton, vary steps via inheritance.

Reach for State when an object has **distinct modes** with different behavior, and it should move between those modes over time — i.e. when you're really modeling a **finite state machine** and want each state's behavior and transitions kept in one cohesive place.